2. Construct the sequence:

$$z_{1:T+1} = (\underbrace{z_1, \ldots, z_{\ell_1}}_{\ell_1 \text{ irrelevant tokens}}, \underbrace{t, o}_{\text{trigger-output pair}}, \underbrace{z_{\ell_1+3}, \ldots, z_{\ell_1+\ell_2+2}}_{\ell_2 \text{ irrelevant tokens}}, \underbrace{t, o}_{\text{trigger-output pair}})$$

where irrelevant token z_i $(i \in [1:\ell_1] \cup [\ell_1+3:\ell_1+\ell_2+2])$ is drawn i.i.d. from $[N_{\text{trg}}+1:N]$.

We refer to such a sequence as a trigger-output *model with subtext lengths ℓ_1 and ℓ_2.*

In our data model, the task is to identify the output token $z_{T+1} = o$ from the sequence $z_{1:T} = (z_1, \ldots, z_{\ell_1},\ t,\ o,\ z_{\ell_1+3}, \ldots, z_{\ell_1+\ell_2+2},\ t)$. This can be achieved by implementing the *induction head mechanism* [ENO+21, OEN+22], which copies the token that follows the first occurrence of the trigger token and outputs it upon encountering the second occurrence of the same trigger.

Due to structure of the input sequence, transformer may also rely on positional shortcuts to achieve low loss; in particular, when the lengths of irrelevant tokens are identical within each sequence, i.e., $\ell_1 = \ell_2 = \ell$, a transformer can achieve 100% training accuracy simply by inferring the correct position to attend to $(T+1)/2 = \ell+2$ from the position of the second trigger $T = 2\ell+3$. Such positional solution does not make use of the semantic information and generally fails when $\ell_1 \neq \ell_2$.

To study the phase transition between the two mechanisms, we assume the pretraining data consists of a mixture of sequences with different lengths determined by $\ell = \ell_1 = \ell_2$.

Definition 2. *Consider a language model $p_{\boldsymbol{\theta}}(\cdot \mid z_1 z_2 \cdots z_T)$ that is pretrained on M sequences $\{z^{(i)}_{1:T^{(i)}+1}\}_{i=1}^M$ generated as follows:*

- *Sample $\ell^{(i)}$ from a distribution $\mathcal{D}_\ell$.*
- *Generate $z^{(i)}_{1:T^{(i)}+1}$ according to Definition 1 with $\ell_1 = \ell_2 = \ell^{(i)}$, i.e.,*

$$z^{(i)}_{1:T^{(i)}+1} = (z_1, \ldots, z_{\ell^{(i)}},\ t,\ o,\ z_{\ell^{(i)}+3}, \ldots, z_{2\ell^{(i)}+2},\ t,\ o).$$

OOD Generalization. Note that the pretraining distribution (defined by $\mathcal{D}_\ell$) may not cover all possible sequences. We say that $p_{\boldsymbol{\theta}}$ *generalizes out-of-distribution (OOD)* if it implements the correct copying mechanism across all possible ℓ's, that is, for any ℓ_1, ℓ_2 such that $\ell_1 + \ell_2 + 3 \leq L - 1$ (possibly $\ell_1 \neq \ell_2$), and for any test sequence $z_{1:T+1}$ generated from the trigger-output unigram model with subtext lengths ℓ_1 and ℓ_2 (Definition 1), we have

$$\arg\max_{k \in [N]} p_{\boldsymbol{\theta}}(k \mid z_1 z_2 \cdots z_T) = z_{T+1}.$$

2.2 Gradient-based Training of Single-layer Transformer

Architecture and Embedding. We consider a single-layer transformer block f_{TF} defined as

$$f_{\text{TF}}(\boldsymbol{X}_{1:t}; \boldsymbol{W}_{KQ}, \boldsymbol{W}_V) = \boldsymbol{W}_V \boldsymbol{X}_{1:t} \operatorname{Softmax}(\boldsymbol{X}_{1:t}^\top \boldsymbol{W}_{KQ} \boldsymbol{x}_t) \in \mathbb{R}^N, \tag{2.1}$$

where $\boldsymbol{W}_{KQ} \in \mathbb{R}^{D \times D}, \boldsymbol{W}_V \in \mathbb{R}^{N \times D}$ and $\boldsymbol{X}_{1:t} = \begin{pmatrix} \boldsymbol{x}_1 & \cdots & \boldsymbol{x}_t \end{pmatrix} \in \mathbb{R}^{D \times t}$ denotes the input embeddings of $z_{1:t}$, with embedding dimension D. We define the embedding as follows:

Definition 3. *Let $D = L + 2N$. Let $\boldsymbol{p}_t \in \mathbb{R}^L$ denote the one-hot vector with a 1 at the t-th position (representing the positional embedding), and let $\boldsymbol{e}_z \in \mathbb{R}^N$ denote the one-hot vector with a 1 at the z-th position (representing the token identity).*

We then construct the input embedding $\boldsymbol{x}_t$ as

$$\boldsymbol{x}_t = \begin{bmatrix} \boldsymbol{p}_t \\ \boldsymbol{e}_{z_t} \\ \boldsymbol{e}_{z_{t-1}} \end{bmatrix} \in \mathbb{R}^{L+2N}. \tag{2.2}$$

Algorithm 1: Gradient-based training of single-layer transformer

Input : Learning rate η_{KQ}, η_V
Initialize $\boldsymbol{W}_{KQ}(0) = \boldsymbol{O}_{(L+2N)\times(L+2N)}, \boldsymbol{W}_V(0) = \boldsymbol{O}_{N\times(L+2N)}$
Gradient descent on $\boldsymbol{W}_V$
$\boldsymbol{W}_V(1) \leftarrow \boldsymbol{W}_V(0) - \eta_V \nabla_{\boldsymbol{W}_V} \frac{1}{M_V} \sum_{i=1}^{M_V} \mathcal{L}(\boldsymbol{X}^{(i)}_{1:T^{(i)}}; \boldsymbol{W}_{KQ}(0), \boldsymbol{W}_V(0))$
Gradient descent on $\boldsymbol{W}_{KQ}$
$\boldsymbol{W}_{KQ}(1) \leftarrow \boldsymbol{W}_{KQ}(0) - \eta_{KQ} \nabla_{\boldsymbol{W}_{KQ}} \frac{1}{M_{KQ}} \sum_{i=M_V+1}^{M_V+M_{KQ}} \mathcal{L}(\boldsymbol{X}^{(i)}_{1:T^{(i)}}; \boldsymbol{W}_{KQ}(0), \boldsymbol{W}_V(1))$
Output: Prediction $f_{\mathrm{TF}}(\cdot)$

The prediction probability is given by

$$p_{(\boldsymbol{W}_{KQ},\boldsymbol{W}_V)}(z_{T+1} = k \mid z_1 \cdots z_T) = [\mathrm{Softmax}(f_{\mathrm{TF}}(\boldsymbol{X}_{1:t}; \boldsymbol{W}_{KQ}, \boldsymbol{W}_V))]_k.$$

Remark 1. *We make the following remarks on the design of our architecture and embedding.*

- *The architecture (with the FFN is absorbed into the value matrix $\boldsymbol{W}_V$, and tied key and query projections) is commonly used in theoretical analyses and mechanistic studies [LLR23, BCB+23, NDL24]; the simplification allows us to focus on the inductive bias by simple attention mechanisms, while retaining sufficient expressiveness to implement algorithmic behaviors.*
- *Two-layer architecture is typically needed to implement the induction head mechanism, where the first layer often learns to detect the trigger and identify of the following token via attention to the previous token [SHT24]. To reflect this inductive step in our simplified single-layer setting, we explicitly encode the identity of the previous token z_{t-1} in the third component of the embedding $\boldsymbol{x}_t$. This choice also echoes recent empirical developments that incorporate information of previous tokens directly into the current state, such as Mamba [GD23], RWKV [PAA+23], and convolution augmentations [LZHO25, All25].*

Gradient-based Learning Algorithm. We use gradient descent (Algorithm 1) on the cross-entropy loss to pretrain our shallow transformer (2.1),

$$\mathcal{L}(\boldsymbol{X}^{(i)}_{1:T^{(i)}}; \boldsymbol{W}_{KQ}, \boldsymbol{W}_V) = \mathrm{CrossEntropy}(\boldsymbol{e}_{z_{T^{(i)}+1}}, \mathrm{Softmax}(f_{\mathrm{TF}}(\boldsymbol{X}_{1:T^{(i)}}; \boldsymbol{W}_{KQ}, \boldsymbol{W}_V))).$$

In Algorithm 1, we apply a *single gradient descent step* with large learning rate on the value and key-query matrices. This is motivated by recent studies [ORST23, BCB+23, WS24] showing that the first gradient step can induce associative memory tied to specific components of the input embedding. In particular, the gradient can often be expressed as a linear combination of outer products $\boldsymbol{w}\boldsymbol{v}^\top$, where either $\boldsymbol{w}$ or $\boldsymbol{v}$ corresponds to embedding vectors such as $\boldsymbol{e}_{z_t}$, $\boldsymbol{e}_{z_{t-1}}$, or $\boldsymbol{p}_t$. Such a gradient structure is sufficient to construct simple forms of associative memory within the model. We remark that similar single-step update is commonly used in the analysis of feature learning in shallow neural networks [BES+22, DLS22, BEG+22] and transformers [OSSW24, NSO+25, WNB+25].

3 Main Result: Data-driven Transition Between Mechanisms

3.1 Positional Shortcut vs. Induction Head

In this section, we illustrate how the diversity of pretraining distribution influences which algorithm the trained transformer implements — either the positional shortcut or the induction head. The following quantity plays a central role in our characterization.

Definition 4. *For each ℓ, let q_ℓ denote the probability mass assigned under $\mathcal{D}_\ell$, and $\mathcal{S}$ the support of $\mathcal{D}_\ell$. We define the max-sum ratio as*

$$R(\mathcal{D}_\ell) = \frac{\max_{\ell\in\mathcal{S}} \ell^{-1} q_\ell}{\sum_{\ell\in\mathcal{S}} \ell^{-1} q_\ell}.$$

Interpretation of max-sum ratio. The max-sum ratio can be seen as a *diversity* measure of $\mathcal{D}_\ell$. The following example provides an intuitive illustration:

Example 1. *Let $\mathcal{D}_\ell = \mathrm{Unif}(\{\ell_0, \ell_0+1, \ldots, \ell_0+K-1\})$. Then the max-sum ratio is given by*

$$R(\ell_0, K) = \frac{\ell_0^{-1}}{\sum_{k=0}^{K-1}(\ell_0+k)^{-1}}, \tag{3.1}$$

which monotonically decreases with K; hence greater diversity of $\mathcal{D}_\ell$ gives smaller max-sum ratio.

Note that the max-sum ratio does not merely capture the width of the distribution: in Example 1, increasing ℓ_0 while keeping K fixed decreases the proportion of ℓ_0^{-1} in $[\ell_0^{-1}, \ldots, (\ell_0+K-1)^{-1}]$, thus reducing the max-sum ratio. Hence, even with a narrow range, shifting the distribution rightward – placing more probability on larger ℓ – naturally yields a smaller max-sum ratio. This is because the max-sum ratio weights each probability mass q_ℓ by ℓ^{-1}.

Learning under Population Loss. The next theorem shows the existence of a threshold in the max-sum ratio that determines whether OOD generalization is achieved, in the infinite-data limit.

Theorem 5 (Infinite Sample Setting)**.** *Suppose we run Algorithm 1 on the expected loss $\mathbb{E}[\mathcal{L}(\boldsymbol{X}_{1:T}; \boldsymbol{W}_{KQ}, \boldsymbol{W}_V)]$ with learning rates $\eta_V \lesssim 1, \eta_V\eta_{KQ} \gtrsim \frac{N^3}{N_{\mathrm{trg}}^3}\log N$. Then, there exist $\epsilon_1(N_{\mathrm{trg}}), \epsilon_2(N_{\mathrm{trg}}) = \Theta(N_{\mathrm{trg}}^{-1})$ such that:*

- *If $R(\mathcal{D}_\ell) < \epsilon_1$, then the pretrained transformer generalizes OOD, as defined in Definition 2.*
- *If $R(\mathcal{D}_\ell) > \epsilon_2$, then there exist OOD test sequences such that the pretrained transformer fails.*

Remark 2.

- *Note that the training data only contain sequences with $\ell_1 = \ell_2$, and thus a positional shortcut (as illustrated in Figure 1) can still achieve 100% training accuracy. However, since the OOD test data include sequences with $\ell_1 \neq \ell_2$, such shortcuts inevitably fail. Our main theorems show that the pretrained transformer avoids such shortcuts when the max-sum ratio is below a certain threshold, i.e., when the data distribution is sufficiently diverse.*
- *We also provide a tight $\Theta(N_{\mathrm{trg}}^{-1})$ characterization of the max-sum ratio threshold, indicating that increasing the number of possible triggers makes OOD generalization more difficult. The underlying mechanism is discussed in the ensuing subsection.*

Learning under Empirical Loss. Our next result establishes (via gradient concentration) similar transition behavior in the finite-sample setting.

Theorem 6 (Finite Sample Setting)**.** *Suppose we run Algorithm 1 with the same learning rate scaling as in Theorem 5, and with sample sizes $M_{KQ} \gtrsim \mathrm{poly}\log N \cdot \frac{N^3}{N_{\mathrm{trg}}^2}\left(\sum_\ell \sqrt{q_\ell}\right)^2$ and $M_V \gtrsim \mathrm{poly}\log N \cdot \frac{N^5}{N_{\mathrm{trg}}^2}\left(\frac{\sum_{\ell\in\mathcal{S}}\sqrt{q_\ell}}{\sum_{\ell\in\mathcal{S}} q_\ell \ell^{-1}}\right)^2$. Then, with high probability there exist $\epsilon_1'(N_{\mathrm{trg}}), \epsilon_2'(N_{\mathrm{trg}}) = \Theta(N_{\mathrm{trg}}^{-1})$ such that the assertion of Theorem 5 holds by substituting $(\epsilon_1', \epsilon_2')$ for (ϵ_1, ϵ_2).*

3.2 Mechanism of Algorithm Selection

Now we take a closer look at how the *positional shortcut* and the *induction head* are implemented in the attention. We begin with the case where the support of $\mathcal{D}_\ell$ is a singleton and $N_{\mathrm{trg}} = 1$. After a single gradient step, the parameter matrix $\boldsymbol{W}_{KQ}$ can be shown to implement a form of associative memory over the relevant embedding vectors.

Lemma 7 (Informal)**.** *Let $\mathcal{D}_\ell = \{\ell\}$, and assume the trigger consists of a single token w. After one gradient step of Algorithm 1, $\boldsymbol{W}_{KQ}$ takes the form*

$$\boldsymbol{W}_{KQ} \propto T(\ell)^{-1}\begin{bmatrix}(\boldsymbol{p}_{\ell+2}+\boldsymbol{p}_{\ell+3}) \\ \boldsymbol{0} \\ \boldsymbol{e}_w\end{bmatrix}\begin{bmatrix}\boldsymbol{p}_{T(\ell)}^\top & \boldsymbol{e}_w^\top & \boldsymbol{0}\end{bmatrix},$$